OptGraph

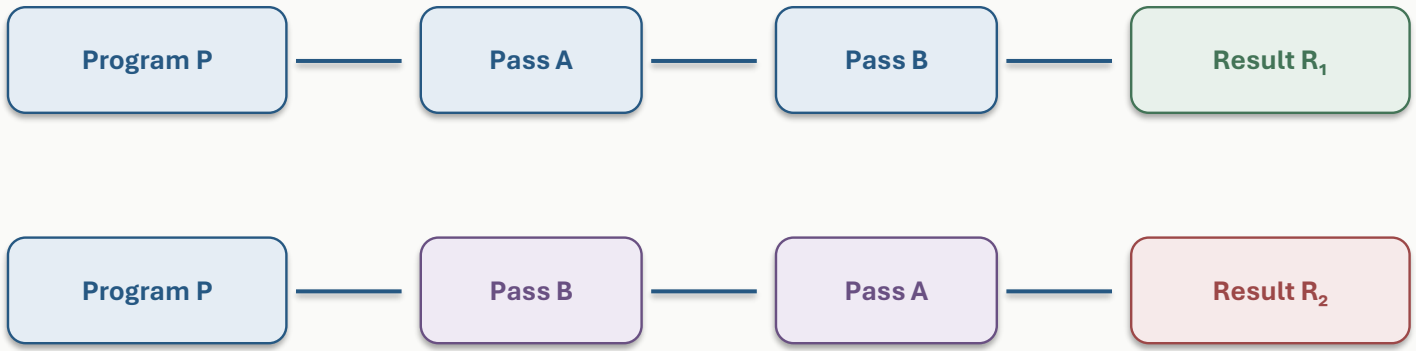
A platform that measures how LLVM optimization passes interact on each program, builds per-program interaction graphs, and predicts them for unseen programs with ML.

Team Members

Arav Singh Patel 24BCE2425
Chaitanya Ghogare 24BCE2392

Problem Identification

The effect of an optimization pass depends on the IR it receives.



$$A \rightarrow B \neq B \rightarrow A$$

Phase Ordering Problem

Why it gets harder

The same pair of passes can also behave differently across programs because loops, control flow, memory operations and data dependencies differ.

Research gap

Can the strength and direction of a pass interaction be explained by program structure, and predicted for unseen programs?

Objective

Measure → explain → predict

01 QUANTIFY

Measure how Pass A changes the benefit of Pass B.

$$I(A,B,P) = \Delta(B|A,P) - \Delta(B|P)$$

02 REPRESENT

Encode synergy / antagonism as weighted graph relationships.

Passes → weighted edges

03 PREDICT

Use static LLVM IR features to predict interactions on unseen programs.

Features + A + B → I

Research questions

- How frequently do standard LLVM passes show measurable synergy or antagonism?
- Which structural features best predict interaction strength?
- Can the model generalize to a completely unseen program?

Literature Survey — ML & Phase Ordering

MILEPOST GCC (2008–2011)

Uses machine learning and program features to adapt compiler optimization heuristics.

Contribution:
• Established program-characterization + ML as a compiler optimization direction.
• Focus: choosing profitable optimizations / heuristics.

Gap for OptGraph:
It does not formulate the target as the interaction strength between a specific pair of passes.

LLVM MLGO

LLVM integrates learned models into compiler heuristics, including inlining-for-size and register-allocation decisions.

Contribution:
• Demonstrates practical ML integration inside LLVM.
• Shows compiler decisions can be learned from compiler features.

Gap for OptGraph:
The target is a compiler decision/heuristic — not a program-conditioned pairwise interaction score.

LLVMTuner / Phase-Ordering Work

Recent systems search or predict effective optimization choices/sequences using LLVM IR features and performance feedback.

Contribution:
• Addresses the enormous phase-ordering search space.
• Demonstrates predictive / search-based optimization tuning.

Gap for OptGraph:
Sequence quality is the outcome; pairwise interaction between A and B is not the primary prediction target.

Sources: Fursin et al., MILEPOST GCC; LLVM MLGO documentation; LLVMTuner (IEEE CSCWD 2025).

Literature Survey — Pass Dependence & Empirical Studies

Efficient Compiler Optimization by Modeling Passes Dependence (2024)

Liu et al. construct an Optimization Dependence Graph (ODG) using source-code dependence and pairwise performance dependence. They use the graph to cluster passes into subsequences for iterative compilation.

What this establishes:
- Pass dependence can be measured empirically.
- Graphs are useful for reasoning about pass relationships.

What OptGraph changes:
- The interaction is explicitly conditioned on program P.
- The goal is prediction on unseen programs, not only graph-based sequence construction.

Multi-Dimensional Per-Pass LLVM Study (2026)

Bruzzone & Cazzola evaluate 113 cumulative -O3 prefixes across 30 PolyBench/C kernels and measure runtime, compile time, binary size, hardware counters and energy.

Key lesson:
- Optimization effects are non-monotone and program-dependent.
- IR instruction count alone is not a reliable runtime proxy.

Research opportunity:
Move from measuring observed pipeline effects to learning a reusable mapping from program structure → pairwise interaction.

Literature synthesis: the field can SEARCH sequences and MEASURE dependence; OptGraph asks whether pairwise interaction itself can be CONDITIONED and PREDICTED.

Sources: Liu et al., CCF Transactions on HPC, 2024; Bruzzone & Cazzola, arXiv:2606.31238, 2026.

Implementation — Interaction Measurement

For each program P and pass pair (A,B), isolate the effect of ordering.





Interaction score

$$I(A,B,P) = \Delta(B \mid A,P) - \Delta(B \mid P)$$

Antagonism (I < 0)
A removes or changes opportunities that B would otherwise exploit.

Independence (I ≈ 0)
A has little measurable effect on B's marginal benefit.

Synergy (I > 0)
A creates / exposes opportunities that make B more beneficial.

Measure separately: IR changes • runtime/cycles • binary size • compilation time

Implementation — Features, Graph & Prediction

Program / IR Features

Static features extracted before the target interaction:

- Instruction count & instruction mix
- Basic blocks, CFG edges & density
- Loop count / maximum loop depth
- Branches & PHI nodes
- Loads, stores & memory ratio
- Calls & arithmetic intensity
- SSA(static single assignment, i.e. every variable is assigned a value exactly once) / IR(Intermediate Representation) structural statistics

Program-Conditioned OIG

For a fixed program P:

Nodes = optimization passes
Edges = pairwise interactions
Weight = I(A,B,P)
Sign = synergy / antagonism
Metadata = confidence + metric

Compare graphs across program families to reveal which relationships are stable and which are context-sensitive.

Prediction on Unseen Code

Model input:
[program features + pass A + pass B]

Models:
- Linear / Logistic baseline
- Random Forest
- XGBoost / Gradient Boosting

Evaluation:
- MAE(Mean Absolute error) / RMSE(Root mean square error)
- Pearson / Spearman correlation
- Sign classification F1
- Program-level holdout

Critical rule: split by program, not individual measurements.

W1 Pipeline | W2 Features | W3 Dataset | W4 Graphs | W5 ML | W6 Eval + Paper

Novelty

The novelty is the prediction target — not the existence of pass interaction.

Prior direction	Primary target	Program-conditioned	Unseen pair interaction
ML-guided optimization	Choose / rank passes or sequences	Often	Not the core target
Pass dependence / ODG	Graph / sequence construction	Partly / empirical	Not the core target
Per-pass empirical studies	Measure observed effects	Yes, experimentally	No predictive model
OPTGRAPH	**Pairwise interaction score**	**YES — explicit**	**YES — primary objective**

Core contribution

I(A,B,P) is treated as a function of the pass pair AND the structural context of program P, then learned for unseen programs.

Novelty Score is 9.3-9.5

Precise claim: program-conditioned quantification + prediction of pairwise optimization-pass interaction.

Expected Research Contribution

1 **Controlled experiments**

2 **Interaction dataset**

3 **PC-OIG graphs**

4 **ML prediction on unseen code**

5 **Compiler insights**

Deliverables

Automated LLVM experimentation harness
Interaction dataset + PC-OIG visualizations
Prediction models + evaluation
Paper-ready results

Syllabus alignment

LLVM / IR • basic blocks • CFG
Data-flow • loops • machine-independent optimization
SSA • code generation context

Research takeaway

Not only:
“Which sequence is best?”

But:
“Why does A change B’s effect — and can we predict it from program structure?”

Program structure → pass interaction → predictive compiler insight